

Catégorisation automatique d'articles de presse en arabe

De la classification classique au service en production : traitement du langage naturel arabe, apprentissage automatique et mise en service via une API.

NIVEAU 1^{re} année cycle ingénieur	DURÉE 5 semaines (+ 3 optionnelles)
TYPE Conception, prototypage & mise en service	DOMAINE NLP · Machine Learning · Data

classification de texte

NLP arabe

TF-IDF

embeddings

AraBERT

multi-label

API REST

Docker

1 Contexte et problématique

Une rédaction en ligne produit et archive de grands volumes d'articles qu'il faut **classer par rubrique** (politique, économie, société, culture...). Fait manuellement, ce classement est coûteux, lent et parfois incohérent d'un rédacteur à l'autre.

La question de ce projet : **peut-on apprendre automatiquement à prédire la ou les rubriques d'un article à partir de son seul texte**, puis exposer ce modèle comme un **service réutilisable** ? Trois difficultés rendent le sujet stimulant : la **langue arabe** (morphologie riche, variantes orthographiques), le caractère **multi-étiquette** (un article peut relever de plusieurs rubriques) et le **déséquilibre** entre catégories.

L'IDÉE — DU TEXTE À LA RUBRIQUE, AUTOMATIQUEMENT

Construire une chaîne de bout en bout qui transforme le texte brut d'un article en une prédiction de rubrique, puis la met à disposition via une API.



2 Objectifs

Objectif général : concevoir, entraîner, évaluer et mettre en service un modèle de classification automatique d'articles de presse en arabe, en comparant plusieurs approches et en le rendant accessible via une API.

- **O1.** Constituer et explorer le jeu de données (analyse, nettoyage, statistiques descriptives).
- **O2.** Mettre en place un **prétraitement du texte arabe** (normalisation, stop-words, racinisation).
- **O3.** Établir une **base de référence** (TF-IDF + classifieur) et l'**évaluer rigoureusement** (validation croisée, matrice de confusion, F1 macro, déséquilibre).
- **O4.** Améliorer via des **représentations vectorielles** (embeddings) et comparer à la base de référence.
- **O5.** **Mettre en service** le modèle via une API (seuil de confiance).
- **Extensions (si le temps le permet) :** modèle transformeur (AraBERT), classification multi-étiquette, explicabilité, conteneurisation Docker.

3 Une montée en compétences progressive

Le projet est structuré comme une **rampe** : il démarre sur des acquis solides (Python, scikit-learn) puis introduit, étape par étape, des concepts nouveaux. Les 5 étapes « **Cœur** » constituent le socle du projet ($\approx$ 5 semaines) ; les 3 étapes « **Extension** » sont **optionnelles**, à approfondir si le temps le permet.

ÉTAPE	CE QUI EST RÉALISÉ	CONCEPT NOUVEAU	PRIORITÉ
Base de référence	TF-IDF + classifieur scikit-learn	chaîne NLP, vectorisation	Cœur
NLP arabe	normalisation (ألف/همزة/تاء), stop-words, racinisation	NLP arabe nouveau	Cœur
Évaluation	validation croisée, matrice de confusion, F1 macro, déséquilibre	évaluation approfondie nouveau	Cœur
Représentations	passage aux <i>embeddings</i> (fastText AR / sentence-transformers)	plongements lexicaux nouveau	Cœur
Mise en service	API FastAPI + seuil de confiance	service ML nouveau	Cœur
Apprentissage profond	comparaison avec un AraBERT affiné	transformeurs nouveau	Extension
Multi-étiquette	plusieurs rubriques par article	multi-label nouveau	Extension
Explicabilité + Docker	LIME + conteneurisation	explicabilité / déploiement nouveau	Extension

4 Données fournies

Un **corpus d'articles de presse en arabe déjà classés par rubrique** est fourni : pas de collecte de données à faire, on commence immédiatement. Les fichiers sont dans le dossier **donnees/**, à côté de cette fiche :

FICHIER	RÔLE	UTILE À	COMMENT L'UTILISER
DATASET.md	Documentation : schéma, conseils (arabe, déséquilibre, multi-label), découpage train/test	À lire en 1 ^{er}	Le lire <i>avant</i> d'écrire du code — il décrit les données et les pièges à éviter.
articles.sample.json	Petit échantillon (15 articles) pour mettre au point le code	Phases 0–1	Le charger pour écrire et tester le pipeline (nettoyage → TF-IDF → 1 ^{er} classifieur) tout de suite. Trop petit pour entraîner un vrai modèle.
articles.json	Corpus complet (milliers d'articles étiquetés) — fourni par l'encadrant	Phases 1–3	Le vrai jeu d'entraînement/évaluation. Se charge exactement comme l'échantillon (même schéma).
export_articles.sh	Script ayant servi à générer articles.json (pour information)	<i>outil encadrant</i>	Tu n'as pas à l'exécuter : articles.json t'est fourni directement.

Schéma d'un article (identique dans l'échantillon et le corpus complet) :

```
{
  "id": "art-0001",
  "title": "...",
  "content": "...", // texte de l'article -> variables (features)
  "categories": ["اقتصاد", "سياسة"], // 1..N rubriques -> etiquettes (labels)
  "publishedAt": "2026-03-04"
}
```

Pour démarrer avec les données — pas à pas

1. Lire donnees/DATASET.md.
2. Charger l'échantillon et l'explorer en Python :

```
import json, pandas as pd
data = json.load(open("donnees/articles.sample.json", encoding="utf-8"))
df = pd.DataFrame(data) # colonnes : id, title, content, categories, publishedAt
print(df.shape)
print(df.head())
```

3. Écrire le pipeline (prétraitement → TF-IDF → classifieur) et le **tester d'abord sur l'échantillon**.

4. Charger le **corpus complet** `articles.json` (fourni par l'encadrant), **de la même façon** que l'échantillon — il contient des milliers d'articles :

```
data = json.load(open("donnees/articles.json", encoding="utf-8"))
df = pd.DataFrame(data)
```

5. Entraîner et évaluer sur `articles.json` ; garder un jeu de **test** à part pour la matrice de confusion.

content = le texte à classer (features) ; **categories** = ce qu'on prédit (labels). Un article pouvant avoir plusieurs rubriques, on commence en *single-label* (prendre `categories[0]`) avant d'aborder le *multi-label*.

5 Travail demandé — phases

PHASE	INTITULÉ	CONTENU
0	Cadrage & exploration	Bibliographie, prise en main des outils, exploration du jeu de données, prétraitement de l'arabe.
1	Base de référence & évaluation	TF-IDF + classifieur, validation croisée, matrice de confusion, F1 macro, gestion du déséquilibre.
2	Amélioration	Représentations par embeddings, comparaison chiffrée à la base de référence.
3	Mise en service & restitution	API FastAPI (endpoint + seuil de confiance), rapport et présentation.
opt.	Extensions	AraBERT, classification multi-étiquette, explicabilité (LIME), conteneurisation Docker.

6 Livrables

1. Notebook(s) d'analyse exploratoire du corpus.
2. Pipeline de prétraitement du texte arabe (réutilisable).
3. Modèles entraînés + **rapport comparatif chiffré** (base de référence vs embeddings vs AraBERT).
4. Évaluation complète : matrices de confusion, F1 par classe, analyse d'erreurs.
5. **API** de prédiction de rubrique (conteneurisée) + documentation d'utilisation.
6. Rapport final + présentation.

7 Technologies

- **Langage & ML** : Python, Pandas, scikit-learn, (extension) HuggingFace Transformers / PyTorch.
- **NLP arabe** : CAMEL Tools ou Farasa (normalisation, racinisation) ; fastText / sentence-transformers.
- **Explicabilité** : LIME (ou importances de caractéristiques).
- **Service** : FastAPI, Docker.
- **Outils** : Jupyter, Git/GitHub.

8 Planning prévisionnel (8 semaines)

SEMAINE	ACTIVITÉ	PHASE
1	Cadrage, exploration des données, prétraitement de l'arabe	0
2	Base de référence TF-IDF + protocole d'évaluation	1
3	Évaluation rigoureuse (validation croisée, F1 macro, déséquilibre)	1
4	Amélioration par embeddings + comparaison	2
5	Mise en service (API) + rapport & présentation	3
6	<i>(option)</i> AraBERT + comparaison	opt.
7	<i>(option)</i> Multi-étiquette + explicabilité (LIME)	opt.
8	<i>(option)</i> Conteneurisation Docker + finalisation	opt.

Le socle du projet tient sur 5 semaines ; les semaines 6 à 8 (extensions) sont optionnelles. Un point d'avancement toutes les deux semaines.

9 Prérequis & compétences visées

Prérequis : Python, bases de l'apprentissage automatique (scikit-learn), notions de statistiques, Git.
Atout : maîtrise de l'arabe.

Compétences développées

- Chaîne complète de **NLP** et de classification de texte.
- **NLP arabe** : normalisation, racinisation, représentations vectorielles.
- **Apprentissage profond** appliqué (transformeurs, AraBERT).

- Évaluation rigoureuse, multi-étiquette, gestion du déséquilibre, explicabilité.
- **Mise en service** d'un modèle (API, Docker) — premiers pas MLOps.

10 Critères d'évaluation

CRITÈRE	PONDÉRATION
État de l'art, exploration & prétraitement des données	15 %
Base de référence + rigueur de l'évaluation	30 %
Amélioration (embeddings) & comparaison	20 %
Mise en service (API)	20 %
Rapport & présentation	15 %
<i>Extensions (AraBERT, multi-label, explicabilité, Docker)</i>	bonus
Total	100 %